Week 2 Module – C Programming :--

Date :-- 21st June 2026

1. Linear Search
Definition:--

Linear Search is the simplest searching technique in which each element of the array is checked one by one until the required element is found or the end of the array is reached. It is mainly used for unsorted arrays.

* Algorithm:--

1.Start from the first element of the array.
2.Compare each element with the target element.
3.If the element matches, display its index and stop.
4.If the end of the array is reached without finding the element, display "Not Found".

* Time Complexity:--

Average Case: $O(n)$

* Uses :--

1. Searching in unsorted arrays.
2. Small datasets.
3. Simple and easy to implement.

* Example :--

Array: 101 105 113 110 130

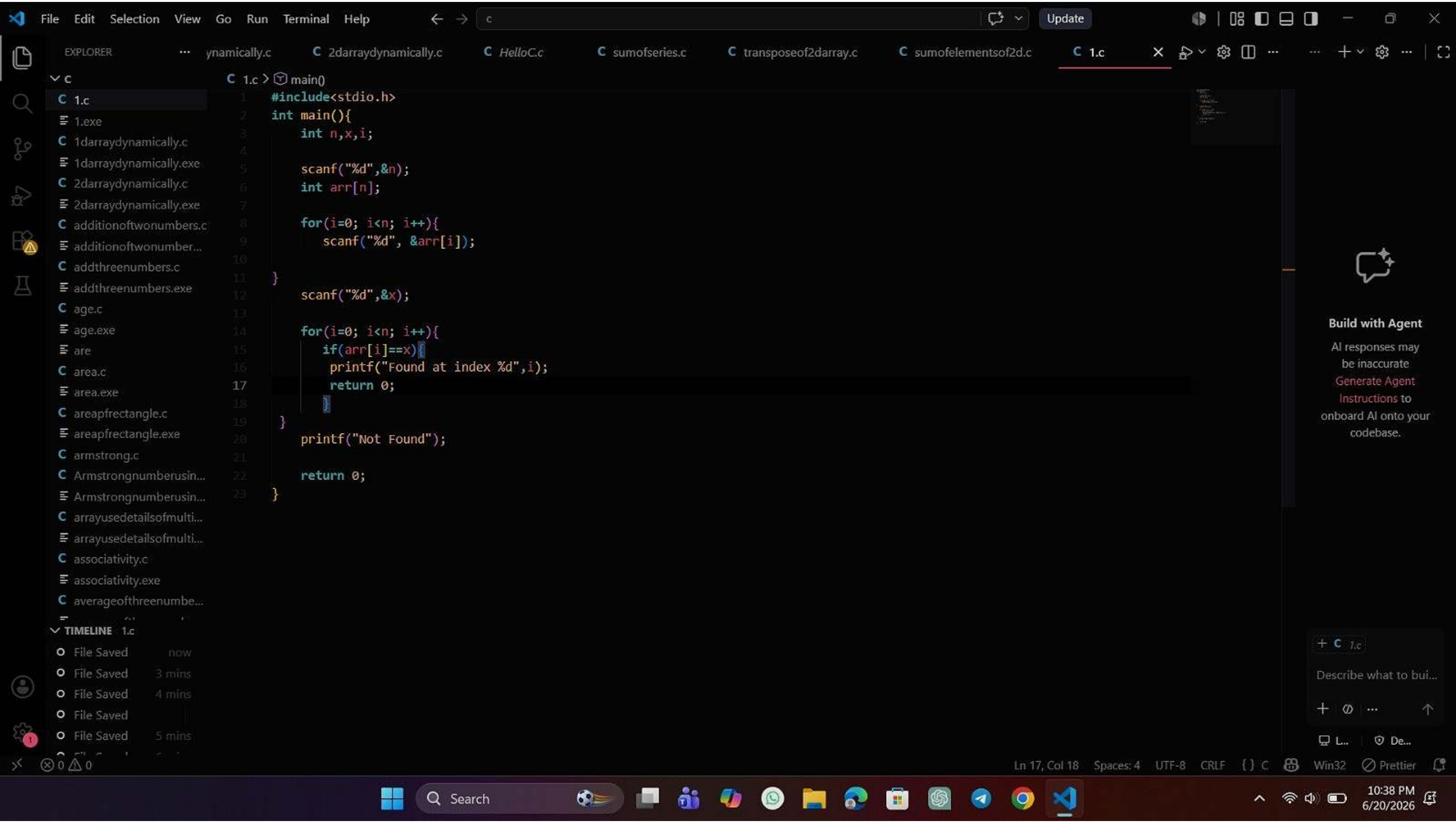
Search Element: 110

Output:

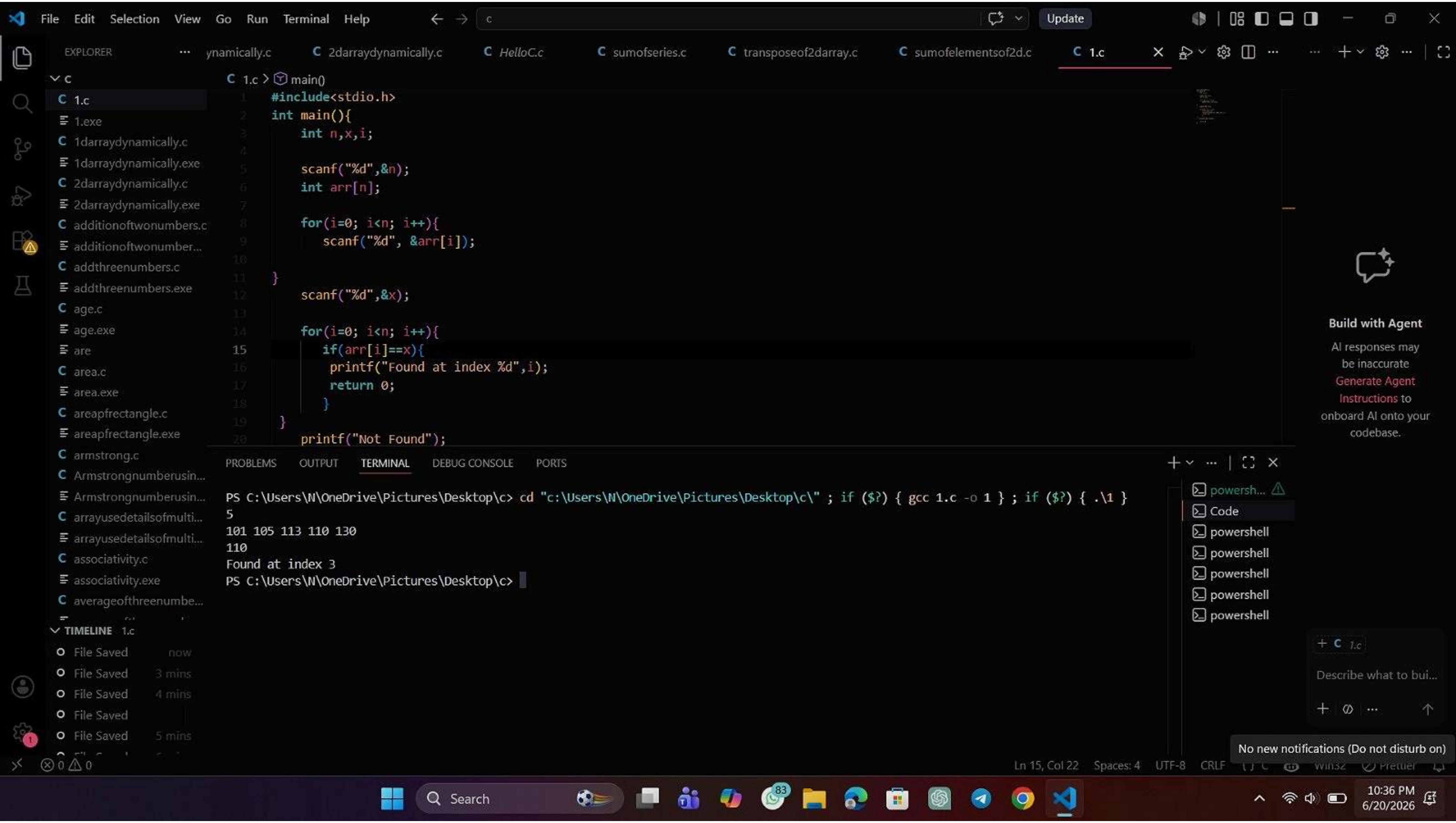
Found at index 3

```
1.c ‡-- #include <stdio.h>

int main() {
int n, x, i;

scanf("%d", &n);

int a[n];

for(i = 0; i < n; i++)
scanf("%d", &a[i]);

scanf("%d", &x);

for(i = 0; i < n; i++) {
if(a[i] == x) {
printf("Found at index %d", i);
return 0;
}
}

printf("Not Found");
```

```
return 0;
}
```



File Edit Selection View Go Run Terminal Help
EXPLORER
1.c
#include<stdio.h>
int main(){
int n,x,i;
scanf("%d",&n);
int arr[n];
for(i=0; i<n; i++){
scanf("%d", &arr[i]);
}
scanf("%d",&x);
for(i=0; i<n; i++){
if(arr[i]==x){
printf("Found at index %d",i);
return 0;
}
}
printf("Not Found");
return 0;
}
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Ln 17, Col 18 Spaces: 4 UTF-8 CRLF C Win32 Prettier
10:38 PM
6/20/2026



File Edit Selection View Go Run Terminal Help
EXPLORER
1.c
1.c > main()
#include<stdio.h>
int main(){
    int n,x,i;

    scanf("%d",&n);
    int arr[n];

    for(i=0; i<n; i++){
       scanf("%d", &arr[i]);

}
    scanf("%d",&x);

    for(i=0; i<n; i++){
       if(arr[i]==x){
        printf("Found at index %d",i);
        return 0;
       }
 }
    printf("Not Found");
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS C:\Users\N\OneDrive\Pictures\Desktop\c> cd "c:\Users\N\OneDrive\Pictures\Desktop\c\" ; if ($?) { gcc 1.c -o 1 } ; if ($?) { .\1 }
5
101 105 113 110 130
110
Found at index 3
PS C:\Users\N\OneDrive\Pictures\Desktop\c>
Build with Agent
No new notifications (Do not disturb on)
Ln 15, Col 22 Spaces: 4 UTF-8 CRLF
10:36 PM
6/20/2026

2. Binary Search.

*Definition:--

Binary Search is a searching technique that works only on sorted arrays. It repeatedly divides the search interval into two halves until the required element is found.

*Algorithm:--

1.Set low = 0 and high = n - 1.
2.Find the middle element.
3.If the middle element is equal to the target, stop.
4.If the target is greater, search the right half.
5.Otherwise, search the left half.
6.Repeat until the element is found or the search interval becomes empty.

*Time Complexity:--

Best Case: O(1)

*Uses:--

1.Searching in sorted arrays.
2.Database searching.
3 Fast searching in large datasets.

*Example:--

Array: 10 20 30 40 50 60
Search Element: 40

Output:
Product Available..

2.c ‡-- #include <stdio.h>

```
int main() {
int n, key, low, high, mid;

scanf("%d", &n);

int a[n];

for(int i = 0; i < n; i++)
scanf("%d", &a[i]);

scanf("%d", &key);

low = 0;
high = n - 1;

while(low <= high) {
mid = (low + high) / 2;

if(a[mid] == key) {
printf("Product Available");
return 0;
```

```
}
else if(key > a[mid])
low = mid + 1;
else
high = mid - 1;
}

printf("Product Not Available");

return 0;
}
```

```c
    int n, target;
    scanf("%d", &n);

    int arr[n];
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);

    int low = 0, high = n - 1, found = 0;

    while(low <= high) {
        int mid = (low + high) / 2;

        if(arr[mid] == target) {
            found = 1;
            break;
        }
        else if(arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    if(found)
        printf("Product Available");
    else
        printf("Product Not Available");

    return 0;
}
```

```c
    int n, target;
    scanf("%d", &n);

    int arr[n];
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);

    int low = 0, high = n - 1, found = 0;

    while(low <= high) {
        int mid = (low + high) / 2;

        if(arr[mid] == target) {
            found = 1;
            break;
        }
        else if(arr[mid] < target)
```

```
PS C:\Users\N\OneDrive\Pictures\Desktop\c> cd "c:\Users\N\OneDrive\Pictures\Desktop\c\" ; if ($?) { gcc 2.c -o 2 } ; if ($?) { .\2 }
6
10 20 30 40 50 60
40
Product Available
PS C:\Users\N\OneDrive\Pictures\Desktop\c>
```

3. Bubble Sort :--

*Definition:--

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order. The process continues until the array is sorted.

*Algorithm:--

1.Compare adjacent elements.
2.Swap them if they are in the wrong order.
3.Repeat the process for all elements.
4.Continue passes until no swaps are needed.

*Time Complexity:--

Best Case: $O(n)$
Average Case: $O(n^2)$

*Uses:--

1.Small datasets.
2.Educational purposes.
3.Easy to understand and implement.

Example:--

Input:
89 45 78 12 99

Output:
12 45 78 89 99

```c
3.c ‡-- #include <stdio.h>

int main() {
int n;

scanf("%d", &n);

int a[n];

for(int i = 0; i < n; i++)
scanf("%d", &a[i]);

for(int i = 0; i < n - 1; i++) {
for(int j = 0; j < n - i - 1; j++) {
if(a[j] > a[j + 1]) {
int temp = a[j];
a[j] = a[j + 1];
a[j + 1] = temp;
}
}
}

for(int i = 0; i < n; i++)
```

```
printf("%d ", a[i]);

return 0;
}
```

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
```

```
PS C:\Users\N\OneDrive\Pictures\Desktop\c> cd "c:\Users\N\OneDrive\Pictures\Desktop\c\" ; if ($?) { gcc 3.c -o 3 } ; if ($?) { .\3 }
5
89 45 78 12 99
12 45 78 89 99
PS C:\Users\N\OneDrive\Pictures\Desktop\c> cd "c:\Users\N\OneDrive\Pictures\Desktop\c\" ; if ($?) { gcc 3.c -o 3 } ; if ($?) { .\3 }
```

4. Insertion Sort:--

*Definition:--

Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position among the already sorted elements.

*Algorithm:--

1.Assume the first element is sorted.
2.Pick the next element.
3.Compare it with previous elements.
4.Shift larger elements one position to the right.
5.Insert the element in its correct position.
6.Repeat for all elements.

*Time Complexity:--

Best Case: $O(n)$
Average Case: $O(n^2)$

*Uses:--

1.Small datasets.
2.Nearly sorted arrays.
3.Online sorting (sorting data as it arrives).

Example:-

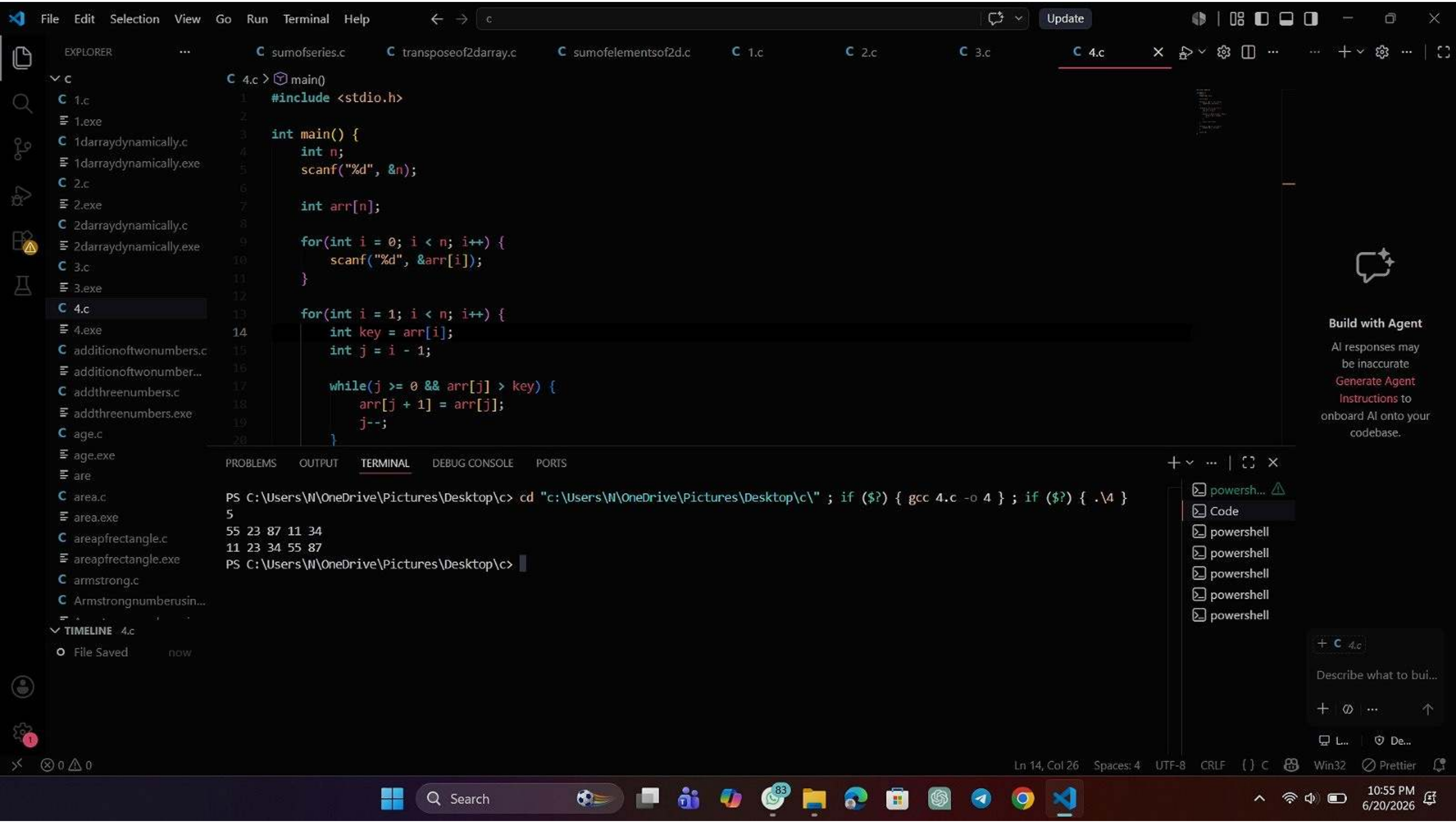
Input:
55 23 87 11 34

Output:
11 23 34 55 87

```
4.c ‡-- #include <stdio.h>

int main() {
int n;

scanf("%d", &n);

int a[n];

for(int i = 0; i < n; i++)
scanf("%d", &a[i]);

for(int i = 1; i < n; i++) {
int key = a[i];
int j = i - 1;

while(j >= 0 && a[j] > key) {
a[j + 1] = a[j];
j--;
}

a[j + 1] = key;
}
```

```
for(int i = 0; i < n; i++)
printf("%d ", a[i]);

return 0;
}
```

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while(j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }

    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```


File Edit Selection View Go Run Terminal Help
Update
EXPLORER
sumofseries.c
transposeof2darray.c
sumofelementsof2d.c
1.c
2.c
3.c
4.c
4.c > main()
#include <stdio.h>
int main() {
int n;
scanf("%d", &n);
int arr[n];
for(int i = 0; i < n; i++) {
scanf("%d", &arr[i]);
}
for(int i = 1; i < n; i++) {
int key = arr[i];
int j = i - 1;
while(j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j];
j--;
}
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS C:\Users\N\OneDrive\Pictures\Desktop\c> cd "c:\Users\N\OneDrive\Pictures\Desktop\c\" ; if ($?) { gcc 4.c -o 4 } ; if ($?) { .\4 }
5
55 23 87 11 34
11 23 34 55 87
PS C:\Users\N\OneDrive\Pictures\Desktop\c>
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to bui...
TIMELINE 4.c
File Saved now
Ln 14, Col 26 Spaces: 4 UTF-8 CRLF { } C Win32 Prettier
Search
10:55 PM
6/20/2026

5. Selection Sort :--

* Definition:--

Selection Sort repeatedly selects the smallest element from the unsorted part of the array and places it at the beginning.

* Algorithm :--

1. Find the smallest element in the unsorted array.
2. Swap it with the first unsorted element.
3. Move the sorted boundary one position forward.
4. Repeat until the entire array is sorted.

* Time Complexity:--

Best Case: $O(n^2)$
Average Case: $O(n^2)$

* Uses:--

1. Small datasets.
2. Situations where the number of swaps should be minimized.
3. Educational purposes.

* Example:--

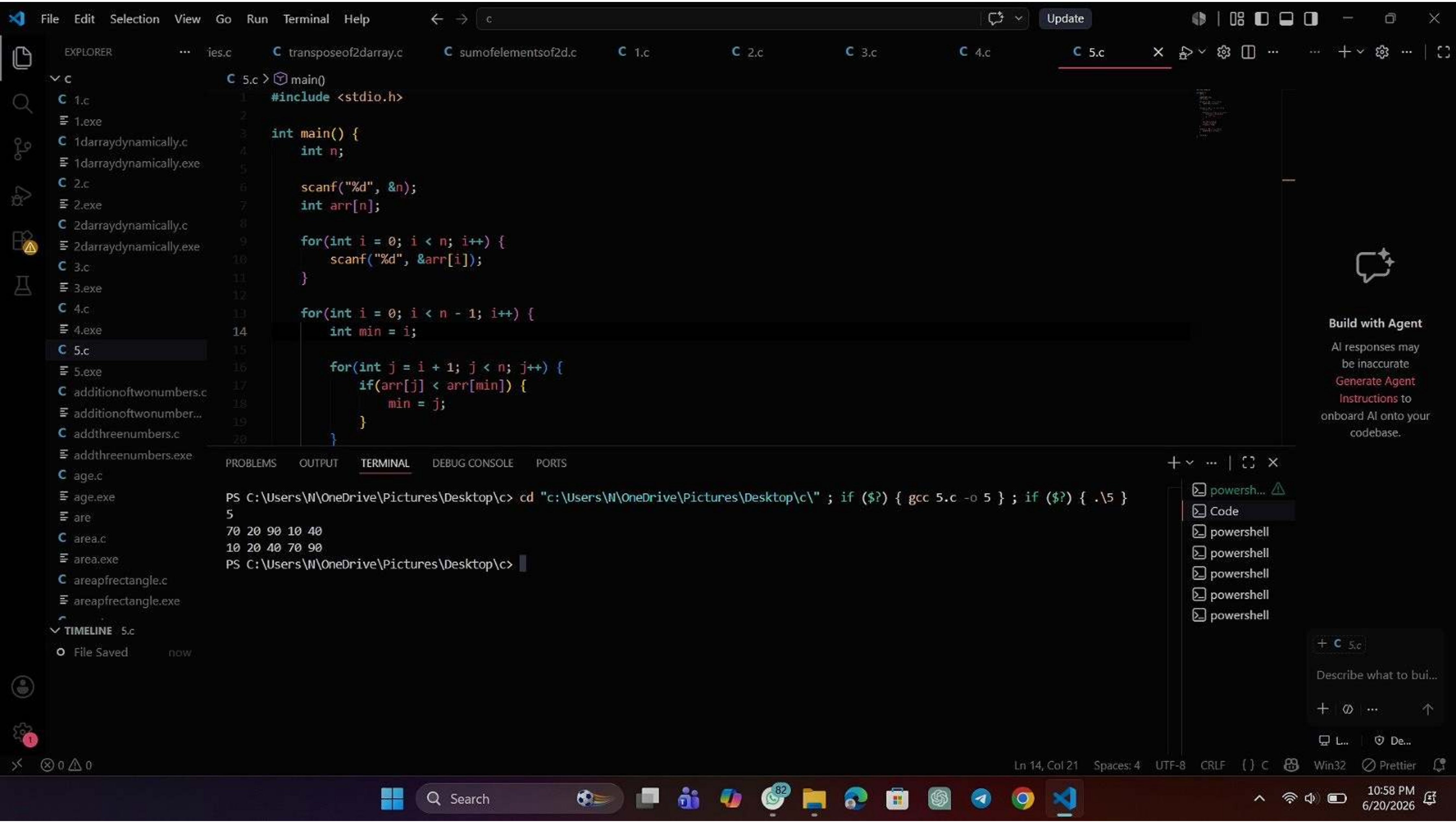
Input:
70 20 90 10 40

Output:
10 20 40 70 90.

```
s.c ‡-- #include <stdio.h>

int main() {
int n;

scanf("%d", &n);

int a[n];

for(int i = 0; i < n; i++)
scanf("%d", &a[i]);

for(int i = 0; i < n - 1; i++) {
int min = i;

for(int j = i + 1; j < n; j++) {
if(a[j] < a[min])
min = j;
}

int temp = a[i];
a[i] = a[min];
a[min] = temp;
```

```
}

for(int i = 0; i < n; i++)
printf("%d ", a[i]);

return 0;
}
```

```c
#include <stdio.h>

int main() {
    int n;

    scanf("%d", &n);
    int arr[n];

    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(int i = 0; i < n - 1; i++) {
        int min = i;

        for(int j = i + 1; j < n; j++) {
            if(arr[j] < arr[min]) {
                min = j;
            }
        }

        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }

    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```



File Edit Selection View Go Run Terminal Help
Update
EXPLORER
transposeof2darray.c
sumofelementsof2d.c
1.c
2.c
3.c
4.c
5.c
5.c > main()
#include <stdio.h>
int main() {
int n;
scanf("%d", &n);
int arr[n];
for(int i = 0; i < n; i++) {
scanf("%d", &arr[i]);
}
for(int i = 0; i < n - 1; i++) {
int min = i;
for(int j = i + 1; j < n; j++) {
if(arr[j] < arr[min]) {
min = j;
}
}
c
1.c
1.exe
1darraydynamically.c
1darraydynamically.exe
2.c
2.exe
2darraydynamically.c
2darraydynamically.exe
3.c
3.exe
4.c
4.exe
5.c
5.exe
additionoftwonumbers.c
additionoftwonumber...
addthreenumbers.c
addthreenumbers.exe
age.c
age.exe
are
area.c
area.exe
areapfrectangle.c
areapfrectangle.exe
TIMELINE 5.c
File Saved now
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS C:\Users\N\OneDrive\Pictures\Desktop\c> cd "c:\Users\N\OneDrive\Pictures\Desktop\c\" ; if ($?) { gcc 5.c -o 5 } ; if ($?) { .\5 }
5
70 20 90 10 40
10 20 40 70 90
PS C:\Users\N\OneDrive\Pictures\Desktop\c>
powersh...
Code
powershell
powershell
powershell
powershell
powershell
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to bui...
Ln 14, Col 21 Spaces: 4 UTF-8 CRLF { } C Win32 Prettier
Search
10:58 PM
6/20/2026